

1. Kafka应用实例

- 1.1. Kafka的Python操作
- 1.2. Kafka与MySQL的组合使用
- 1.3. Kafka采集数据保存到MongoDB中

2. Flume应用实例

- 2.1 采集NetCat数据显示到控制台
- 2.2 采集目录下的数据显示到控制台
- 2.3. Flume和Kafka的组合使用
 - 2.3.1 Flume采集NetCat数据到Kafka
 - 2.3.2 Flume采集文件数据到Kafka
 - 2.3.2 Flume采集MySQL数据到Kafka
- 2.4. 采集日志文件到HDFS
 - 2.4.1 采集目录到HDFS
 - 2.4.2 采集文件到HDFS
- 2.5. 采集MySQL数据到HDFS
- 2.6. Flume多数据源应用实例

1. Kafka应用实例

Kafka的安装与使用

Kafka的运行需要Java环境的支持，因此，安装Kafka前需要在Ubuntu操作系统中安装JDK。请参照第2章内容完成JDK的安装。访问Kafka官网

（<https://kafka.apache.org/downloads>），下载Kafka3.6.1的安装文件kafka_2.12-3.6.1.tgz。在Ubuntu的终端中执行如下命令安装Kafka：

```
$ cd /home/hadoop/Downloads #假设安装文件在该目录下
$ sudo tar -zxvf kafka_2.12-3.6.1.tgz -C /usr/local
$ cd /usr/local
$ sudo mv kafka_2.12-3.6.1 kafka
$ sudo chown -R hadoop ./kafka
```

首先需要启动Kafka。请在Ubuntu系统中打开一个终端，输入下面命令启动Zookeeper服务：

```
$ cd /usr/local/kafka
$ ./bin/zookeeper-server-start.sh config/zookeeper.properties
```

请另外打开第二个终端，然后输入下面命令启动Kafka服务：

```
$ cd /usr/local/kafka
$ ./bin/kafka-server-start.sh config/server.properties
```

下面先测试一下Kafka是否可以正常使用。再打开第三个终端，然后输入下面命令创建一个自定义名称为“wordsendertest”的Topic：

```
$ cd /usr/local/kafka
$ ./bin/kafka-topics.sh --create --topic wordsendertest --
bootstrap-server localhost:9092
#可以用list列出所有创建的Topic，来查看上面创建的Topic是否存在
$ ./bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

```
hadoop@vm:/usr/local/kafka$ ./bin/kafka-topics.sh --list --bootstrap-server l
localhost:9092
__consumer_offsets
test
wordsendertest
```



下面用生产者（Producer）来产生一些数据，请在当前终端内继续输入下面命令：

```
$ ./bin/kafka-console-producer.sh --bootstrap-server
localhost:9092 --topic wordsendertest
```

上面命令执行后，就可以在当前终端（假设名称为“生产者终端”）内用键盘输入一些英文单词，比如可以输入：

```
hello hadoop
hello kafka
```

```
hadoop@vm:/usr/local/kafka$ ./bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --top
ic wordsendertest
>hello hadoop
>hello kafka
>
```

请另外打开第四个终端，输入下面命令：

```
$ cd /usr/local/kafka
$ ./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092
--topic wordsendertest --from-beginning
```

可以看到，屏幕上会显示出如下结果，也就是刚才在另外一个终端里面输入的内容：

```
hadoop@vm:/usr/local/kafka$ ./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic wordsendertest --from-beginning
hello hadoop
hello kafka
```

1.1. Kafka的Python操作

在使用Python操作Kafka之前，需要安装第三方模块kafka-python，命令如下：

```
$ pip install kafka-python
```

安装结束以后，可以使用如下命令查看已经安装的kafka-python的版本信息：

```
$ pip list
```

执行这个命令后显示已经安装的Python第三方模块，以及每个模块的版本。

```
itemloaders 1.4.0
jeepney 0.7.1
jmespath 1.1.0
kafka-python 2.3.1
keyring 23.5.0
language-selector 0.1
launchpadlib 1.10.16
lazr.restfulclient 0.14.4
lazr.uri 1.0.6
lockfile 0.12.2
```



编写一个生产者程序producer_test.py用来生成消息：

```
# producer_test.py

from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers='localhost:9092') # 连接Kafka

msg = "Hello world".encode('utf-8') # 发送内容，必须是bytes类型
producer.send('test', msg) # 发送的Topic为test
producer.close()
```

编写一个消费者程序consumer_test.py用来消费消息：

```
# consumer_test.py

from kafka import KafkaConsumer

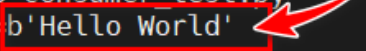
consumer=KafkaConsumer('test', bootstrap_servers=
['localhost:9092'], group_id=None, auto_offset_reset='smallest')
for msg in consumer:
    recv = "%s:%d:%d: key=%s value=%s" % (msg.topic,
msg.partition, msg.offset, msg.key, msg.value)
    print(recv)
```

在Ubuntu终端中启动Zookeeper服务和Kafka服务，然后，先执行producer_test.py，再执行consumer_test.py，具体命令如下：

```
$ cd /home/hadoop #假设代码在该目录下
$ python3 producer_test.py
$ python3 consumer_test.py
```

如果程序执行成功，可以看到屏幕上显示“Hello World”。

```
hadoop@vm:~/code$ python3 consumer_test.py
test:0:0: key=None value=b'Hello World'
```



面再给出一个稍微复杂一点的实例。假设“/home/hadoop”目录下有一个文件score.csv，其内容如下：

```
"Name","Score"
"Zhang San",99.0
"Li Si",45.5
"Wang Hong",82.5
"Liu Qian",76.0
"Ma Li",62.5
"Shen Teng",78.0
"Pu Wen",86.5
```

用excel打开如下如所示：

	A	B	C	D
1	Name	Score		
2	Zhang San	99		
3	Li Si	45.5		
4	Wang Hon	82.5		
5	Liu Qian	76		
6	Ma Li	62.5		
7	Shen Teng	78		
8	Pu Wen	86.5		
9				
10				

要求完成的任务是，Kafka生产者读取文件中的所有内容，然后，以JSON字符串的形式发送给Kafka消费者，消费者获得消息以后转换成表格形式打印到屏幕上，如下所示：

	Name	Score
0	Zhang San	99.0
1	Li Si	45.5
2	Wang Hong	82.5
3	Liu Qian	76.0
4	Ma Li	62.5
5	Shen Teng	78.0
6	Pu Wen	86.5

为了完成上述任务，可以编写代码文件kafka_demo.py（要求和文件score.csv在同一个目录下），其内容如下：

```
# kafka_demo.py

import sys
import json
import pandas as pd
from kafka import KafkaProducer
from kafka import KafkaConsumer
from kafka.errors import KafkaError

KAFKA_HOST = "localhost" #服务器地址
KAFKA_PORT = 9092        #端口号
KAFKA_TOPIC = "topic0"   #topic

data=pd.read_csv("score.csv")
key_value=data.to_json()
```

```

class kafka_producer():
    def __init__(self, kafkahost, kafkaport, kafkatopic, key):
        self.kafkaHost = kafkahost
        self.kafkaPort = kafkaport
        self.kafkatopic = kafkatopic
        self.key = key
        self.producer =
KafkaProducer(bootstrap_servers='{kafka_host}:{kafka_port}'.format(
    kafka_host=self.kafkaHost,
    kafka_port=self.kafkaPort)
)
    def sendjsondata(self, params):
        try:
            params_message = params
            producer = self.producer
            producer.send(self.kafkatopic, key=self.key,
value=params_message.encode('utf-8'))
            producer.flush()
        except KafkaError as e:
            print(e)

class kafka_consumer():
    def __init__(self, kafkahost, kafkaport, kafkatopic,
groupid,key):
        self.kafkaHost = kafkahost
        self.kafkaPort = kafkaport
        self.kafkatopic = kafkatopic
        self.groupid = groupid
        self.key = key
        self.consumer = KafkaConsumer(self.kafkatopic,
group_id=self.groupid,
    auto_offset_reset='earliest',
    bootstrap_servers='{kafka_host}:{kafka_port}'.format(
        kafka_host=self.kafkaHost,
        kafka_port=self.kafkaPort)
    )
    def consume_data(self):
        try:
            for message in self.consumer:
                yield message
        except KeyboardInterrupt as e:

```

```

        print(e)

def sortedDictValues(adict):
    items = adict.items()
    items=sorted(items,reverse=False)
    return [value for key, value in items]

def main(xtype, group, key):
    if xtype == "p":
        # 生产模块
        producer = Kafka_producer(KAFKA_HOST, KAFKA_PORT,
KAFKA_TOPIC, key)
        print("=====> producer:", producer)
        params =key_value
        producer.sendjsondata(params)
    if xtype == 'c':
        # 消费模块
        consumer = Kafka_consumer(KAFKA_HOST, KAFKA_PORT,
KAFKA_TOPIC, group,key)
        print("=====> consumer:", consumer)
        message = consumer.consume_data()
        for msg in message:
            msg=msg.value.decode('utf-8')
            python_data=json.loads(msg) ##字符串转换成字典
            key_list=list(python_data)
            test_data=pd.DataFrame()
            for index in key_list:
                if index=='Name':
                    a1=python_data[index]
                    data1 = sortedDictValues(a1)

                    test_data[index]=data1
                else:
                    a2 = python_data[index]
                    data2 = sortedDictValues(a2)
                    test_data[index] = data2
            print(test_data)

if __name__ == '__main__':
    main(xtype='p',group='py_test',key=None)
    main(xtype='c',group='py_test',key=None)

```

在终端中执行如下命令运行程序（运行结果如图所示）：

```
$ cd /home/hadoop
$ python3 kafka_demo.py
```

```
hadoop@vm:~/code$ python3 kafka_demo.py
=====> producer: <__main__.Kafka_producer object at 0x7c52ed20c0d0>
=====> consumer: <__main__.Kafka_consumer object at 0x7c52ed20c0d0>
      Name  Score
0  Zhang San  99.0
1    Li Si   45.5
2  Wang Hong  82.5
3  Liu Qian  76.0
4    Ma Li   62.5
5  Shen Teng  78.0
6    Pu Wen  86.5
```

1.2. Kafka与MySQL的组合使用

本节通过一个实例来演示Kafka与MySQL的组合使用。需要完成的任务：把JSON格式数据放入Kafka发送出去；然后从Kafka中获取JSON格式数据，对其进行解析并写入MySQL数据库。请参照第2章的内容完成MySQL的安装，并学习其使用方法。

编写一个生产者程序producer_json.py：

```
# producer_json.py

from kafka import KafkaProducer
import json

producer =
KafkaProducer(bootstrap_servers='localhost:9092',value_serializer=1
lambda v:json.dumps(v).encode('utf-8')) # 连接kafka

data={
    "sno":"95001",
    "name":"John",
    "sex":"M",
    "age":23
}

producer.send('json', data) # 发送的topic为json
```



```
producer.close()
```

编写一个消费者程序consumer_json.py:

```
# consumer_json.py

from kafka import KafkaConsumer
import json
import pymysql.cursors

consumer = KafkaConsumer('json', bootstrap_servers=
['localhost:9092'],group_id=None,auto_offset_reset='earliest')
for msg in consumer:
    msg1=str(msg.value, encoding = "utf-8")
    dict = json.loads(msg1)
    # 连接数据库
    connect = pymysql.Connect(
        host='localhost',
        port=3306,
        user='root', # 数据库用户名
        passwd='123456', # 密码
        db='school',
        charset='utf8'
    )

    # 获取游标
    cursor = connect.cursor()

    # 插入数据
    sql = "INSERT INTO student(sno,sname,ssex,sage) VALUES ('%s',
'%s', '%s', %d)"
    data = (dict['sno'],dict['name'],dict['sex'],dict['age'])
    cursor.execute(sql % data)
    connect.commit()
    print('成功插入数据')

    # 关闭数据库连接
    connect.close()
```

在Ubuntu系统中启动MySQL服务进程，然后，进入MySQL Shell界面，输入如下SQL语句创建数据库school:

```
mysql> CREATE DATABASE school;
```

创建好数据库school以后，可以使用如下SQL语句打开数据库:

```
mysql> USE school;
```

使用如下SQL语句创建一个表student:

```
mysql> CREATE TABLE student(  
-> sno char(5),  
-> sname char(10),  
-> ssex char(2),  
-> sage int);
```

使用如下SQL语句查看已经创建的表:

```
mysql> SHOW TABLES;  
mysql> DESCRIBE student;
```

结果如图所示:

```
mysql> show tables;  
+-----+  
| Tables_in_school |  
+-----+  
| student          |  
+-----+  
1 row in set (0.01 sec)  
  
mysql> DESCRIBE student;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type      | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| sno   | char(5)   | YES  |     | NULL    |       |  
| sname | char(10)  | YES  |     | NULL    |       |  
| ssex  | char(2)   | YES  |     | NULL    |       |  
| sage  | int       | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

在Ubuntu终端中启动Zookeeper服务和Kafka服务。然后，先执行producer_json.py，再执行consumer_json.py，具体命令如下：

```
$ cd /home/hadoop #假设代码在该目录下
$ python3 producer_json.py
$ python3 consumer_json.py
```

执行成功以后，使用如下命令查看MySQL数据库中新插入的记录：

```
mysql> SELECT * FROM student;
```

可以看到，一条记录已经被成功地插入MySQL数据库。如图所示：

```
mysql> SELECT * FROM student;
+-----+-----+-----+-----+
| sno   | sname | ssex  | sage |
+-----+-----+-----+-----+
| 95001 | John  | M     | 23   |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

1.3. Kafka采集数据保存到MongoDB中

任务描述：

在Ubuntu系统的“/home/hadoop”目录下有一个stat.csv文件，里面包含如下两行内容：

```
1,2025-1-21,Ziyu,Lin
2,2025-1-22,Shufan,Lin
```

用excel打开查看，如图所示：

	A	B	C	D	E	F
1	1	2025/1/21	Ziyu	Lin		
2	2	2025/1/22	Shufan	Lin		
3						
4						
5						
6						

现在需要编写程序，使用Kafka采集stat.csv中的数据，并对数据进行解析，然后保存到MongoDB数据库中。

在“/home/hadoop”目录下新建一个代码文件producer.py，其内容如下：

```
# producer.py

from kafka import KafkaProducer

print("this is producer")
producer = KafkaProducer(bootstrap_servers=['localhost:9092'],
api_version=(0, 10))
csvFilePath = 'stat.csv'

data = []
with open(csvFilePath, "rb") as csvfile:
    data = csvfile.readlines()
    for rec in data:
        # Topic为'csvDataTopic' 消息内容为读取的CSV文件的一行
        producer.send('csvDataTopic', rec)
        print(rec)
        print("发送消息成功")
producer.close()
```

在“/home/hadoop”目录下新建一个代码文件consumer.py，其内容如下：

```
# consumer.py

from kafka import KafkaConsumer
from pymongo import MongoClient

print("this is consumer")
consumer = KafkaConsumer('csvDataTopic',bootstrap_servers=
['localhost:9092'],auto_offset_reset='earliest')
client = MongoClient(host='localhost', port=27017)
db = client.db

result = {}
csv_data = []
header_arr = ['id','timestamp','first_name','last_name']

for message in consumer:
    # 收到的订阅消息处理
```

```
msg_str=message.value.decode('utf-8').strip().replace('\n', '')
csv_data = msg_str.split(',')
print(csv_data)
data = {header_arr[i] : str(csv_data[i]) for i in
range(len(header_arr))}
result = db.csvstats.insert_one(data)
```

在Ubuntu系统中启动Zookeeper服务和Kafka服务（参考4.5.2节内容），启动MongoDB服务（参考第2.6节内容）。新建一个终端，执行如下命令启动生产者和消费者：

```
$ cd /home/hadoop
$ python3 producer.py
$ python3 consumer.py
```

新建一个终端，执行如下命令进入MongoDB Shell执行环境：

```
$ mongosh
```

然后，在MongoDB Shell环境中执行如下命令查看数据：

```
>show dbs;
>use db;
>show collections;
>db.csvstats.find();
```

查询结果如图所示：

```
test> show dbs;
admin    40.00 KiB
config   60.00 KiB
db        40.00 KiB
local    72.00 KiB
test> use db;
switched to db db
db> show collections;
csvstats
db> db.csvstats.find();
[
  {
    _id: ObjectId( '6a0f9d07a9576f6d26f6db28' ),
    id: '1',
    timestamp: '2025-1-21',
    first_name: 'Ziyu',
    last_name: 'Lin'
  },
  {
    _id: ObjectId( '6a0f9d07a9576f6d26f6db29' ),
    id: '2',
    timestamp: '2025-1-22',
    first_name: 'Shufan',
    last_name: 'Lin'
  }
]
db> █
```

2. Flume应用实例

Flume的安装与配置

Flume的运行需要Java环境的支持，因此，需要在Ubuntu系统中安装JDK。请参照第2章内容完成JDK的安装。访问Flume官网（<https://flume.apache.org/download.html>），下载Flume安装文件apache-flume-1.11.0-bin.tar.gz。

打开一个Ubuntu终端，执行如下命令安装Flume：

```
$ cd /home/hadoop/Downloads # 假设Flume安装文件在该目录下
$ sudo tar -zxvf apache-flume-1.11.0-bin.tar.gz -C /usr/local
$ cd /usr/local
$ sudo mv apache-flume-1.11.0-bin flume
$ sudo chown -R hadoop ./flume # hadoop是当前登录Ubuntu系统的用户名
```

执行如下命令测试是否安装成功：

```
$ cd /usr/local/flume
$ ./bin/flume-ng version
```

如果能够返回类似如下图的信息，则表示安装成功：

```
hadoop@vm:/usr/local$ cd /usr/local/flume
hadoop@vm:/usr/local/flume$ ./bin/flume-ng version
Flume 1.11.0
Source code repository: https://git.apache.org/repos/asf/flume.git
Revision: 1a15927e594fd0d05a59d804b90a9c31ec93f5e1
Compiled by rgoers on Sun Oct 16 14:44:15 MST 2022
From source with checksum bbbca682177262aac3a89defde369a37
hadoop@vm:/usr/local/flume$
```

执行如下命令设置Flume运行时的日志显示级别：

```
$ cd /usr/local/flume/conf
$ vim log4j2.xml
```

使用vim编辑器打开log4j2.xml文件以后，在文件中找到“”这个配置项的位置，并配置为如下内容：

```
<Root level="INFO">
  <AppenderRef ref="Console" />
  <AppenderRef ref="LogFile" />
</Root>
```

```
<Loggers>
  <Logger name="org.apache.flume.lifecycle" level="info"/>
  <Logger name="org.jboss" level="WARN"/>
  <Logger name="org.apache.avro.ipc.netty.NettyTransceiver" level="WARN"/>
  <Logger name="org.apache.hadoop" level="INFO"/>
  <Logger name="org.apache.hadoop.hive" level="ERROR"/>
  <Root level="INFO">
    <AppenderRef ref="Console" />
    <AppenderRef ref="LogFile" />
  </Root>
</Loggers>
```

在上图里面的配置信息中，“”这行是新增的内容，其他内容是log4j2.xml文件中原本就存在的信息。

2.1 采集NetCat数据显示到控制台

这里给出一个简单的实例，假设Source为NetCat类型，使用Uubuntu自带的nc命令连接Source写入数据，产生的日志数据输出到控制台（屏幕）。

在Ubuntu系统的“/usr/local/flume/conf”目录下，新建一个配置文件example.conf，内容如下：

```
# example.conf

# Name the components on this agent
a1.sources = r1
a1.sinks = k1
a1.channels = c1

# source
a1.sources.r1.type = netcat
a1.sources.r1.bind = localhost
a1.sources.r1.port = 44444

# sink
a1.sinks.k1.type = logger

# channel
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

配置文件设置了Source类型为NetCat，Channel类型为Memory，Sink类型为Logger。然后，新建一个终端（这里称为“Flume窗口”），并执行如下命令：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/example.conf --name a1 -Dflume.root.logger=INFO,console
```

再新建一个终端，并执行如下nc命令：


```
$ nc localhost 4444
```

这时就可以从键盘输入一些英文单词，比如“Hello World”（如图所示）。

```
hadoop@vm:~$ nc localhost 4444
Hello World
OK
```

切换到Flume窗口，就可以看到屏幕上显示了“Hello World”，如图5-3所示。这说明Flume成功地接收到了信息。

```
2026-05-22 08:41:42,379 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - o
rg.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{}
body: 48 65 6C 6C 6F 20 57 6F 72 6C 64
Hello World
```

2.2 采集目录下的数据显示到控制台

假设Ubuntu系统中有一个目录“/home/hadoop/mylogs”（不需要新建这个目录），这个目录下不断有新的文件生成。使用Flume采集这个目录下的文件，并把文件内容显示到控制台（屏幕）。

在Flume安装目录的conf子目录下，新建一个名称为example1.conf的配置文件，该文件的内容如下：

```
# example1.conf

# 设置Agent上的各个组件名称
a1.sources = r1
a1.channels = c1
a1.sinks = k1

# 配置Source
a1.sources.r1.type = spooldir
a1.sources.r1.spoolDir = /home/hadoop/mylogs

# 配置Channel
a1.channels.c1.type = memory
a1.channels.c1.capacity = 10000
a1.channels.c1.transactionCapacity = 100

# 配置Sink
```

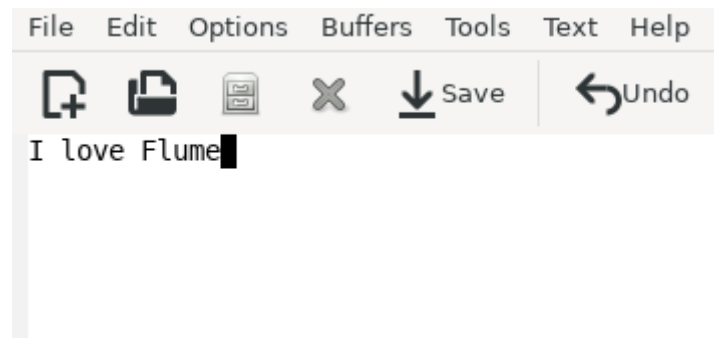
```
a1.sinks.k1.type = logger

# 把Source和Sink绑定到Channel上
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

清空“/home/hadoop/mylogs”目录（即删除该目录下的所有内容），然后新建一个终端（这里称为“Flume窗口”），并执行如下命令：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/example1.conf --name a1 -Dflume.root.logger=INFO,console
```

然后，在“/home/hadoop”目录下新建一个文件mylog.txt，输入一些内容，如“I love Flume”。如图所示：



保存该文件，并把该文件复制到“/home/hadoop/mylogs”目录下，可以看到，mylogs目录下的mylog.txt很快会变成mylog.txt.COMPLETED，这时，在Flume窗口中就可以看到mylog.txt中的内容，如“I love Flume”（如图所示）。

```
2026-05-22 08:52:28,720 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - o
rg.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{}
body: 49 20 6C 6F 76 65 20 46 6C 75 6D 65
I love Flume
```

2.3. Flume和Kafka的组合使用

2.3.1 Flume采集NetCat数据到Kafka

在Ubuntu系统中打开第1个终端，执行如下命令启动Zookeeper服务：

```
$ cd /usr/local/kafka  
$ ./bin/zookeeper-server-start.sh config/zookeeper.properties
```

打开第2个终端，执行如下命令启动Kafka服务：

```
$ cd /usr/local/kafka  
$ ./bin/kafka-server-start.sh config/server.properties
```

打开第3个终端，执行如下命令创建一个名为test的Topic：

```
$ cd /usr/local/kafka  
$ ./bin/kafka-topics.sh --create --topic test --bootstrap-server  
localhost:9092
```

在Flume安装目录的conf子目录下创建一个配置文件kafka.conf，内容如下：

```
# kafka.conf  
  
# Name the components on this agent  
a1.sources = r1  
a1.sinks = k1  
a1.channels = c1  
  
# source  
a1.sources.r1.type = netcat  
a1.sources.r1.bind = localhost  
a1.sources.r1.port = 44444  
  
# sink  
a1.sinks.k1.type = org.apache.flume.sink.kafka.KafkaSink  
a1.sinks.k1.kafka.topic = test  
a1.sinks.k1.kafka.bootstrap.servers = localhost:9092  
a1.sinks.k1.kafka.flumeBatchSize = 20  
a1.sinks.k1.kafka.producer.acks = 1  
a1.sinks.k1.kafka.producer.linger.ms = 1  
a1.sinks.k1.kafka.producer.compression.type = snappy
```

```
# channel
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

打开第4个终端，执行如下命令启动Flume：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file ./conf/kafka.conf
--name a1 -Dflume.root.logger=INFO,console
```

打开第5个终端，执行如下命令：

```
$ nc localhost 44444
```

执行上面的命令以后，在该窗口内用键盘输入一些单词，如“hadoop”。这个单词会发送给Flume，然后由Flume发送给Kafka。如图所示：

```
hadoop@vm:~$ nc localhost 44444
hadoop
OK
```

打开第6个终端，执行如下命令启动Kafka消费者：

```
$ cd /usr/local/kafka
$ ./bin/kafka-console-consumer.sh --bootstrap-server
localhost:9092 --topic test --from-beginning
```

上面的命令执行以后，就可以在屏幕上看到“hadoop”（如图所示），说明Kafka成功接收到了数据。

```
hadoop@vm:/usr/local/kafka$ ./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning
Hello World
hadoop
```

2.3.2 Flume采集文件数据到Kafka

1.方案设计

如下图所示，设置两个Agent，即Agent1和Agent2，二者的组件配置如下：

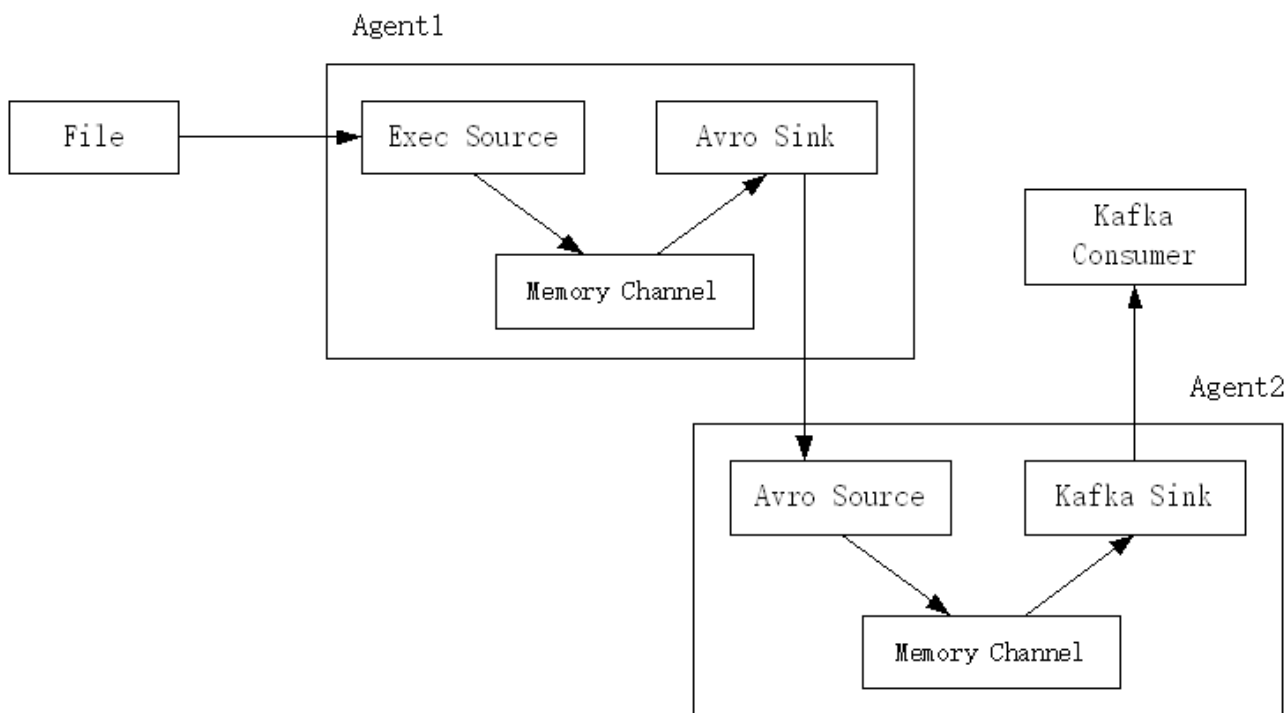
- Agent1: Exec Source + Memory Channel + Avro Sink
- Agent2: Avro Source + Memory Channel + Kafka Sink

在Agent1中，各个组件的功能如下：

- Exec Source：实施监控一个文件的内容是否有增加；
- Avro Sink：用于跨节点传输，把捕捉到的数据传递给Agent2。

在Agent2中，各个组件的功能如下：

- Avro Source：接收来自Agent1的数据；
- Kafka Sink：消费数据。



2.配置Flume

为Agent1创建一个配置文件flume_kafka1.conf，其内容如下：

```
# flume_kafka1.conf

# 设置各组件名称
a1.sources = exec-source
a1.sinks = avro-sink
a1.channels = memory-channel

#配置Source
a1.sources.exec-source.type = exec
a1.sources.exec-source.command = tail -F /home/hadoop/data.txt

#配置Sink
a1.sinks.avro-sink.type = avro
a1.sinks.avro-sink.hostname = localhost
a1.sinks.avro-sink.port = 44444

#配置Channel
a1.channels.memory-channel.type = memory
a1.channels.memory-channel.capacity = 1000
a1.channels.memory-channel.transactionCapacity = 100

#把Source和Sink绑定到Channel
a1.sources.exec-source.channels = memory-channel
a1.sinks.avro-sink.channel = memory-channel
```

注意，在配置文件flume_kafka1.conf中，“tail -F /home/hadoop/data.txt”这个配置信息中一定要使用大写F。

为Agent2创建一个配置文件flume_kafka2.conf，其内容如下：

```
# flume_kafka2.conf

#设置各组件名称
a2.sources = avro-source
a2.sinks = kafka-sink
a2.channels = memory-channel

#配置Source
a2.sources.avro-source.type = avro
```

```
a2.sources.avro-source.bind = localhost
a2.sources.avro-source.port = 44444

# 配置Sink
a2.sinks.kafka-sink.type = org.apache.flume.sink.kafka.KafkaSink
a2.sinks.kafka-sink.kafka.bootstrap.servers = localhost:9092
a2.sinks.kafka-sink.kafka.topic = myTestTopic
a2.sinks.kafka-sink.serializer.class =
kafka.serializer.StringEncoder
a2.sinks.kafka-sink.kafka.producer.acks = 1
a2.sinks.kafka-sink.custom.encoding = UTF-8
a2.sinks.kafka-sink.flumeBatchSize=1

# 配置Channel
a2.channels.memory-channel.type = memory
a2.channels.memory-channel.capacity = 1000
a2.channels.memory-channel.transactionCapacity = 100

# 把Source和Sink绑定到Channel
a2.sources.avro-source.channels = memory-channel
a2.sinks.kafka-sink.channel = memory-channel
```

3. 执行过程

在Ubuntu系统中打开第1个终端，执行如下命令启动Zookeeper服务：

```
$ cd /usr/local/kafka
$ ./bin/zookeeper-server-start.sh config/zookeeper.properties
```

打开第2个终端，执行如下命令启动Kafka服务：

```
$ cd /usr/local/kafka
$ ./bin/kafka-server-start.sh config/server.properties
```

打开第3个终端，执行如下命令创建一个名为myTestTopic的Topic：

```
$ cd /usr/local/kafka
$ ./bin/kafka-topics.sh --create --topic myTestTopic --bootstrap-
server localhost:9092
```

下面需要启动两个Agent，需要先启动Agent2，再启动Agent1，否则，Agent1启动会被拒绝连接。

打开第4个终端，执行如下命令启动Agent2:

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/flume_kafka2.conf --name a2 -Dflume.root.logger=INFO,console
```

打开第5个终端，执行如下命令启动Agent1:

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/flume_kafka1.conf --name a1 -Dflume.root.logger=INFO,console
```

打开第6个终端，执行如下命令启动Kafka消费者消费数据:

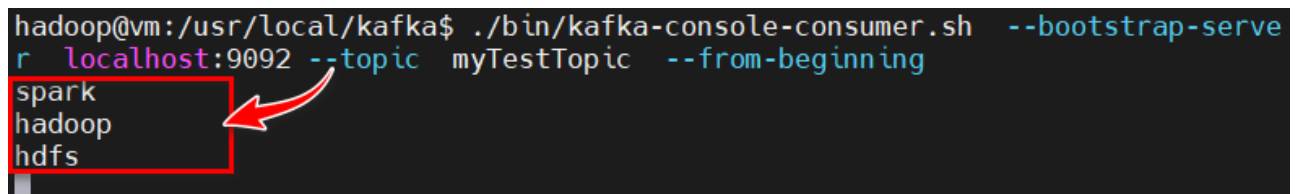
```
$ cd /usr/local/kafka
$ ./bin/kafka-console-consumer.sh --bootstrap-server
localhost:9092 --topic myTestTopic --from-beginning
```

打开第7个终端，输入如下命令向data.txt文件追加数据:

```
$ echo spark >> "/home/hadoop/data.txt"
$ echo hadoop >> "/home/hadoop/data.txt"
$ echo hdfs >> "/home/hadoop/data.txt"
```

这时，在第6个终端，就可以看到捕捉到的数据（如图所示）。

```
hadoop@vm:/usr/local/kafka$ ./bin/kafka-console-consumer.sh --bootstrap-serve
r localhost:9092 --topic myTestTopic --from-beginning
spark
hadoop
hdfs
```



2.3.2 Flume采集MySQL数据到Kafka

1.准备工作

在采集MySQL数据库中的数据到MySQL时，需要用到一个第三方JAR包，即flume-ng-sql-source-1.5.3.jar。此外，为了让Flume能够顺利连接MySQL数据库，还需要用到一个连接驱动程序JAR包，即mysql-connector-java-8.0.30.jar。然后，把flume-ng-sql-source-1.5.3.jar和mysql-connector-java-8.0.30.jar这两个文件复制到Flume安装目录的lib子目录下（比如“/usr/local/flume/lib”）

2.创建数据库

参照第2章中关于MySQL数据库的内容，完成MySQL的安装，并学习其基本使用方法，这里假设读者已经成功安装了MySQL数据库并掌握了基本的使用方法。在Ubuntu系统中，启动MySQL服务进程，然后，打开MySQL的命令行客户端，执行如下SQL语句创建数据库和表：

```
mysql>CREATE DATABASE school;
mysql> USE school;
mysql> CREATE TABLE student(
    -> id INT NOT NULL,
    -> name VARCHAR(40),
    -> age INT,
    -> grade INT,
    -> PRIMARY KEY (id));
```

需要注意的是，在创建表的时候，一定要设置一个主键（比如，这里id是主键），否则后面Flume会捕捉数据失败。

3.配置Flume

进入Flume安装目录下的conf子目录，在里面新建一个文件mysql_kafka.conf，在里面输入如下内容：

```
# mysql_kafka.conf

# 设置Agent上的各个组件名称
a1.channels = ch-1
a1.sources = src-1
a1.sinks = k1

#设置Source组件
```

```

a1.sources.src-1.type = org.keedio.flume.source.SQLSource
a1.sources.src-1.hibernate.connection.url =
jdbc:mysql://localhost:3306/school
a1.sources.src-1.hibernate.connection.user = root
a1.sources.src-1.hibernate.connection.password = 123456
a1.sources.src-1.hibernate.connection.autocommit = true
a1.sources.src-1.table = student      #数据库中的表名称
a1.sources.src-1.run.query.delay=5000
a1.sources.src-1.custom.query = select id, name,age,grade from
student
a1.sources.src-1.status.file.path = /usr/local/flume
a1.sources.src-1.status.file.name = sql-source.status

# 配置Sink
a1.sinks.k1.type = org.apache.flume.sink.kafka.KafkaSink
a1.sinks.k1.kafka.topic = testTopic
a1.sinks.k1.kafka.bootstrap.servers = localhost:9092
a1.sinks.k1.kafka.flumeBatchSize = 20
a1.sinks.k1.kafka.producer.acks = 1
a1.sinks.k1.kafka.producer.linger.ms = 1
a1.sinks.k1.kafka.producer.compression.type = snappy

# 配置Channel
a1.channels.ch-1.type = memory
a1.channels.ch-1.capacity = 1000
a1.channels.ch-1.transactionCapacity = 100

# 把Source和Sink绑定到Channel上
a1.sources.src-1.channels = ch-1
a1.sinks.k1.channel = ch-1

```

配置文件mysql_kafka.conf中有如下两行：

```

a1.sources.src-1.status.file.path = /usr/local/flume
a1.sources.src-1.status.file.name = sql-source.status

```

这两行设置了Flume状态信息的保存位置，即保存在“/usr/local/flume”目录下的sql-source.status这个文件中。需要重点强调的是，sql-source.status这个文件一定不要自己创建（如果自己创建，启动Flume时会报错），Flume在启动过程中会自动创建这个文件。如果已经存在sql-source.status这个文件，也要将其删除。

4.执行过程

在Ubuntu系统中打开第1个终端，执行如下命令启动Zookeeper服务：

```
$ cd /usr/local/kafka
$ ./bin/zookeeper-server-start.sh config/zookeeper.properties
```

打开第2个终端，执行如下命令启动Kafka服务：

```
$ cd /usr/local/kafka
$ ./bin/kafka-server-start.sh config/server.properties
```

打开第3个终端，执行如下命令创建一个名为testTopic的Topic：

```
$ cd /usr/local/kafka
$ ./bin/kafka-topics.sh --create --topic testTopic --bootstrap-server localhost:9092
```

打开第4个终端，执行如下命令启动Flume：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file ./conf/mysql_kafka.conf --name a1 -Dflume.root.logger=INFO,console
```

然后，在MySQL命令行客户端中执行如下语句向MySQL数据库中插入数据：

```
mysql> insert into student(id,name,age,grade)
values(1,'Xiaoming',23,98);
mysql> insert into student(id,name,age,grade)
values(2,'Zhangsan',24,96);
mysql> insert into student(id,name,age,grade)
values(3,'Lisi',24,93);
```

打开第5个终端，执行如下命令启动Kafka消费者：

```
$ cd /usr/local/kafka
$ ./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic testTopic --from-beginning
```

上面命令执行以后，就可以在屏幕上看到如图所示数据。

```
hadoop@vm:/usr/local/kafka$ ./bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic testTopic --from-beginning
"1","Xiaoming","23","98"
"2","Zhangsan","24","96"
"3","Lisi","24","93"
"1","Xiaoming","23","98"
"2","Zhangsan","24","96"
"3","Lisi","24","93"
"1","Xiaoming","23","98"
"2","Zhangsan","24","96"
"3","Lisi","24","93"
"1","Xiaoming","23","98"
"2","Zhangsan","24","96"
"3","Lisi","24","93"
"1","Xiaoming","23","98"
"2","Zhangsan","24","96"
"3","Lisi","24","93"
"1","Xiaoming","23","98"
"2","Zhangsan","24","96"
"3","Lisi","24","93"
```

2.4. 采集日志文件到HDFS

2.4.1 采集目录到HDFS

某服务器的某特定目录下（如“/home/hadoop/mylogs”）会不断产生新的文件，每当有新文件出现，就需要把文件采集到HDFS。

根据需求，首先定义以下三大要素。

1. Source: 因为要监控文件目录，所以Source的类型是spooldir。
2. Sink: 因为要把文件采集到HDFS中，所以，Sink的类型是hdfs。
3. Channel: Channel的类型可以设置为memory。

在Flume安装目录的conf子目录下，编写一个配置文件spooldir_hdfs.conf，其内容如下：

```
# spooldir_hdfs.conf

# 定义三大组件的名称
agent1.sources = source1
agent1.sinks = sink1
agent1.channels = channel1

# 配置Source
agent1.sources.source1.type = spooldir
agent1.sources.source1.spoolDir = /home/hadoop/mylogs/
```

```

agent1.sources.source1.fileHeader = false

# 配置Sink
agent1.sinks.sink1.type = hdfs
agent1.sinks.sink1.hdfs.path = hdfs://localhost:9000/weblog/%y-%m-%d/%H-%M
agent1.sinks.sink1.hdfs.filePrefix = access_log
agent1.sinks.sink1.hdfs.maxOpenFiles = 5000
agent1.sinks.sink1.hdfs.batchSize= 100
agent1.sinks.sink1.hdfs.fileType = DataStream
agent1.sinks.sink1.hdfs.writeFormat =Text
agent1.sinks.sink1.hdfs.rollSize = 102400
agent1.sinks.sink1.hdfs.rollCount = 1000000
agent1.sinks.sink1.hdfs.rollInterval = 60
#agent1.sinks.sink1.hdfs.round = true
#agent1.sinks.sink1.hdfs.roundValue = 10
#agent1.sinks.sink1.hdfs.roundUnit = minute
agent1.sinks.sink1.hdfs.useLocalTimeStamp = true

# 配置Channel
agent1.channels.channel1.type = memory
agent1.channels.channel1.keep-alive = 120
agent1.channels.channel1.capacity = 500000
agent1.channels.channel1.transactionCapacity = 600

# 把Source和Sink绑定到Channel上
agent1.sources.source1.channels = channel1
agent1.sinks.sink1.channel = channel1

```

配置文件spooldir_hdfs.conf中还有如下一行：

```

agent1.sinks.sink1.hdfs.path = hdfs://localhost:9000/weblog/%y-%m-%d/%H-%M

```

这行配置信息设置了数据在HDFS中的保存目录。需要注意的是，这个目录不需要自己创建，Flume会自动在HDFS中创建该目录。

这行配置信息中的“%y-%m-%d/%H-%M”使用了通配符，具体含义如下表所示。更多的通配符及其用法可以参考Flume官网资料。

通配符	含义
%y	年份的最后两位数字（从00到99）
%m	月份（从01到12）
%d	一个月中的第几天（从01到31）
%H	小时（从00到23）
%M	分钟（从00到59）

为了让Flume能够顺利访问HDFS，需要把相关的JAR包复制到Flume安装目录下的lib子目录下，具体命令如下：

```
$ cd /usr/local/flume/lib
$ rm guava-11.0.2.jar # 删除这个JAR包，避免和Hadoop中的同名JAR包发生版本冲突
$ cd /usr/local/hadoop/share/hadoop/hdfs/
$ cp ../*.jar /usr/local/flume/lib
$ cd /usr/local/hadoop/share/hadoop/hdfs/lib
$ cp ../*.jar /usr/local/flume/lib
$ cd /usr/local/hadoop/share/hadoop/common/lib/
$ cp ../*.jar /usr/local/flume/lib
$ cd /usr/local/hadoop/share/hadoop/common/
$ cp ../*.jar /usr/local/flume/lib
```

在Ubuntu系统中，新建一个终端，使用如下命令启动Hadoop的HDFS：

```
$ cd /usr/local/hadoop
$ ./sbin/start-dfs.sh
```

执行JDK自带的命令jps，查看Hadoop已经启动的进程：

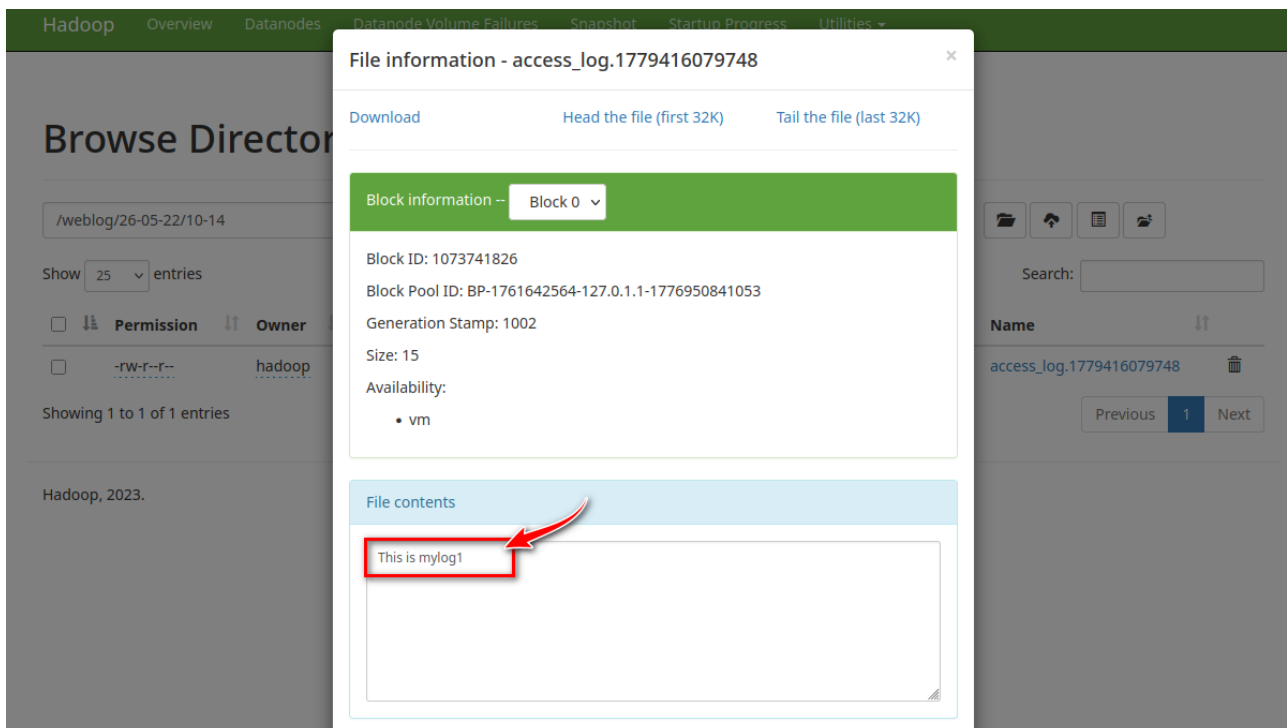
```
$ jps
```

执行jps命令以后，如果能够看到DataNode、NameNode和SecondaryNameNode这3个进程，就说明Hadoop启动成功。

再新建一个终端，执行如下命令启动Flume：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/spooldir_hdfs.conf --name agent1 -
Dflume.root.logger=INFO,console
```

执行上述命令以后，Flume就被启动了，开始实时监控“/home/hadoop/mylogs/”目录（如果mylogs目录不存在，需要手动创建）。只要这个目录下有新的文件生成，就会被Flume捕捉到，并被保存到HDFS中。在“/home/hadoop”目录下新建一个文本文件mylog1.txt，写入一些句子，如“This is mylog1”，然后，把mylog1.txt文件复制到“/home/hadoop/mylogs/”目录下，过一会儿，就会看到mylog1.txt文件名被修改成了mylog1.txt.COMPLETED，说明该文件已经成功被Flume捕捉到。可以在HDFS的Web管理页面（<http://localhost:9870>）中查看生成的文件及其内容（如图所示），如果看到文件扩展名是tmp，比如access_log.1704980778358.tmp，说明文件还在写入过程中，再等待几秒钟，等到写入结束以后，刷新页面，tmp扩展名就会消失，这时就可以查看文件里面的内容。



2.4.2 采集文件到HDFS

某服务器的某特定目录下的文件（如“/home/hadoop/mylogs/log1.txt”）会不断发生更新，每当文件被更新时，就需要把更新的数据采集到HDFS。根据需求，首先定义以下三要素。

1. Source: 因为要监控文件内容，所以Source的类型是exec。
2. Sink: 因为要把文件采集到HDFS，所以Sink的类型是hdfs。
3. Channel: Channel的类型可以设置为memory。

在Flume安装目录的conf子目录下，编写一个配置文件exec_hdfs.conf，其内容如下：

```

# exec_hdfs.conf

# 定义三大组件的名称
agent1.sources = source1
agent1.sinks = sink1
agent1.channels = channel1

# 配置Source
agent1.sources.source1.type = exec
agent1.sources.source1.command = tail -F
/home/hadoop/mylogs/log1.txt
agent1.sources.source1.channels = channel1

# 配置Sink
agent1.sinks.sink1.type = hdfs
agent1.sinks.sink1.hdfs.path =hdfs://localhost:9000/weblog/%y-%m-
%d/%H-%M
agent1.sinks.sink1.hdfs.filePrefix = access_log
agent1.sinks.sink1.hdfs.maxOpenFiles = 5000
agent1.sinks.sink1.hdfs.batchSize= 100
agent1.sinks.sink1.hdfs.fileType = DataStream
agent1.sinks.sink1.hdfs.writeFormat =Text
agent1.sinks.sink1.hdfs.rollSize = 102400
agent1.sinks.sink1.hdfs.rollCount = 1000000
agent1.sinks.sink1.hdfs.rollInterval = 60
#agent1.sinks.sink1.hdfs.round = true
#agent1.sinks.sink1.hdfs.roundValue = 10
#agent1.sinks.sink1.hdfs.roundUnit = minute
agent1.sinks.sink1.hdfs.useLocalTimeStamp = true

# 配置Channel
agent1.channels.channel1.type = memory
agent1.channels.channel1.keep-alive = 120
agent1.channels.channel1.capacity = 500000
agent1.channels.channel1.transactionCapacity = 600

# 把Source和Sink绑定到Channel上
agent1.sources.source1.channels = channel1
agent1.sinks.sink1.channel = channel1

```

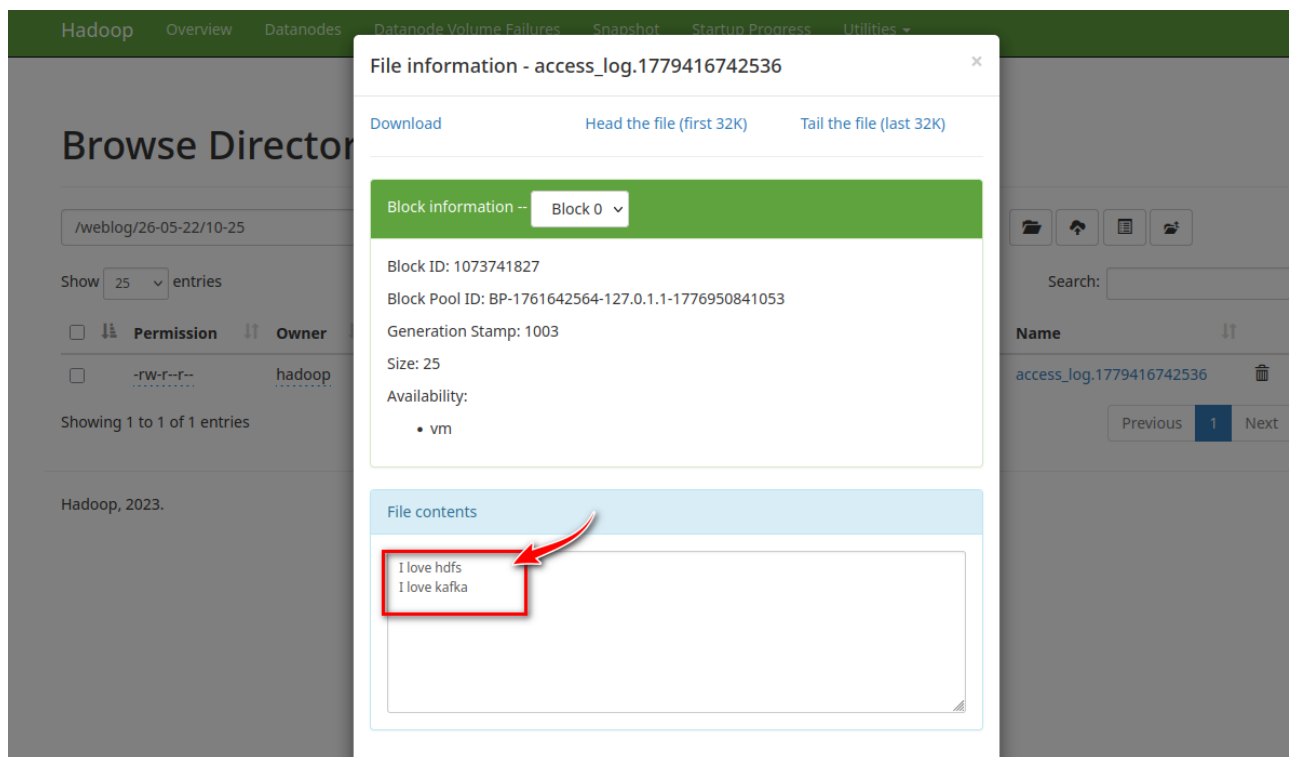
新建一个终端，启动HDFS。然后执行如下命令启动Flume:


```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/exec_hdfs.conf --name agent1 -
Dflume.root.logger=INFO,console
```

执行上述命令以后，Flume就被启动了，开始实时监控“/home/hadoop/mylogs/log1.txt”文件。只要这个文件发生了内容更新，就会被Flume捕捉到，更新内容会被保存到HDFS中。作为测试，可以在log1.txt文件中输入一些内容，

```
$ echo "I love hdfs" >> /home/hadoop/mylogs/log1.txt
$ echo "I love kafka" >> /home/hadoop/mylogs/log1.txt
```

然后到HDFS的Web管理页面中（<http://localhost:9870>）查看生成的文件及其内容。



2.5. 采集MySQL数据到HDFS

准备工作

在前面节中如果已经完成下面的准备工作，则不需要重复操作。在采集MySQL数据库中的数据到MySQL时，需要用到一个第三方JAR包，即flume-ng-sql-source-1.5.3.jar。此外，为了让Flume能够顺利连接MySQL数据库，还需要用到一个连接驱动程序JAR包，即mysql-connector-java-8.0.30.jar。然后，把flume-ng-sql-source-1.5.3.jar和mysql-connector-

java-8.0.30.jar这两个文件复制到Flume安装目录的lib子目录下（比如“/usr/local/flume/lib”）。

创建MySQL数据库

参照第2章中关于MySQL数据库的内容完成MySQL数据库的安装，并学习其基本使用方法。这里假设读者已经成功安装了MySQL数据库并掌握了基本的使用方法。

打开一个终端，进入MySQL Shell界面，执行如下SQL语句创建数据库和表：

```
mysql> DROP DATABASE school;
mysql> CREATE DATABASE school;
mysql> USE school;
mysql> CREATE TABLE student(
    -> id INT NOT NULL,
    -> name VARCHAR(40),
    -> age INT,
    -> grade INT,
    -> PRIMARY KEY (id));
```

需要注意的是，在创建表的时候，一定要设置一个主键（例如，这里id是主键），否则后面Flume会捕捉数据失败。

配置和启动Flume

创建好MySQL数据库以后，再执行如下命令启动Hadoop的HDFS：

```
$ cd /usr/local/hadoop
$ ./sbin/start-dfs.sh
```

根据需求，首先定义以下三要素。

1. Source：因为要监控MySQL数据库，所以Source的类型是org.keedio.flume.source.SQLSource。
2. Sink：因为要把文件采集到HDFS中，所以Sink的类型是hdfs。
3. Channel：Channel的类型可以设置为memory。

在Flume安装目录的conf子目录下，编写一个配置文件mysql_hdfs.conf，其内容如下：

```
# mysql_hdfs.conf
```

```
# 定义三大组件的名称
agent1.channels = ch1
agent1.sinks = HDFS
agent1.sources = sql-source

# 配置Source
agent1.sources.sql-source.type = org.keedio.flume.source.SQLSource
agent1.sources.sql-source.hibernate.connection.url =
jdbc:mysql://localhost:3306/school
agent1.sources.sql-source.hibernate.connection.user = root
agent1.sources.sql-source.hibernate.connection.password = 123456
agent1.sources.sql-source.hibernate.connection.autocommit = true
agent1.sources.sql-source.table = student
agent1.sources.sql-source.run.query.delay=5000
agent1.sources.sql-source.status.file.path = /usr/local/flume
agent1.sources.sql-source.status.file.name = sql-source.status

# 配置Sink
agent1.sinks.HDFS.type = hdfs
agent1.sinks.HDFS.hdfs.path = hdfs://localhost:9000/flume/mysql
agent1.sinks.HDFS.hdfs.fileType = DataStream
agent1.sinks.HDFS.hdfs.writeFormat = Text
agent1.sinks.HDFS.hdfs.rollSize = 268435456
agent1.sinks.HDFS.hdfs.rollInterval = 0
agent1.sinks.HDFS.hdfs.rollCount = 0

# 配置Channel
agent1.channels.ch1.type = memory

# 把Source和Sink绑定到Channel上
agent1.sinks.HDFS.channel = ch1
agent1.sources.sql-source.channels = ch1
```

配置文件mysql_hdfs.conf中有如下两行：

```
agent1.sources.sql-source.status.file.path = /usr/local/flume
agent1.sources.sql-source.status.file.name = sql-source.status
```

这两行设置了Flume状态信息的保存位置，即保存在“/usr/local/flume”目录下的sql-source.status这个文件中。需要重点强调的是，sql-source.status这个文件一定不要自己创建（如果自己创建，启动Flume时会报错），Flume在启动过程中会自动创建这个文件。如果已经存在sql-source.status这个文件，也要将其删除。

执行如下命令启动Flume:

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file
./conf/mysql_hdfs.conf --name agent1 -
Dflume.root.logger=INFO,console
```

执行上述命令以后，Flume就被启动了。一定要注意启动过程中的返回信息，看看是否返回了错误信息。当返回的信息中没有任何错误信息时，就表示启动成功了。

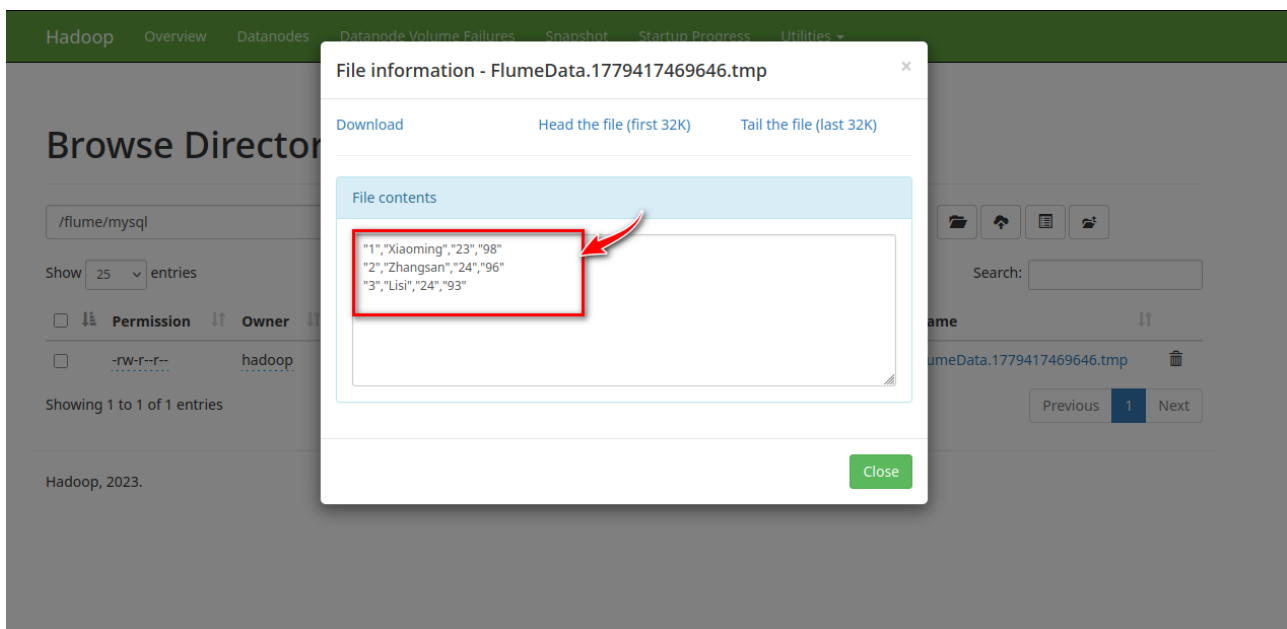
然后，在MySQL Shell界面中执行如下语句向MySQL数据库中插入数据:

```
mysql> insert into student(id,name,age,grade)
values(1,'Xiaoming',23,98);
mysql> insert into student(id,name,age,grade)
values(2,'Zhangsan',24,96);
mysql> insert into student(id,name,age,grade)
values(3,'Lisi',24,93);
```

到“/usr/local/flume”目录下查看sql-source.status这个文件，这个文件会包含类似下面的信息:

```
{"SourceName":"sql-
source","URL":"jdbc:mysql://localhost:3306/school","LastIndex":"
3","ColumnsToSelect":"*","Table":"student"}
```

在浏览器中输入网址“<http://localhost:9870>”打开Hadoop的Web管理界面, 就可以看到新生成的文件。打开其中一个文件，在出现的页面中单击“Tail the file(last 32K)”，就会显示文件的内容,如图所示。



2.6. Flume多数据源应用实例

方案设计

设置三个Agent，即Agent1、Agent2和Agent3，三者的组件配置如下（如图所示）：

- Agent1: Exec Source + Memory Channel + Avro Sink
- Agent2: NetCat Source + Memory Channel + Avro Sink
- Agent3: Avro Source + Memory Channel + Logger Sink

在Agent1中，各个组件的功能如下：

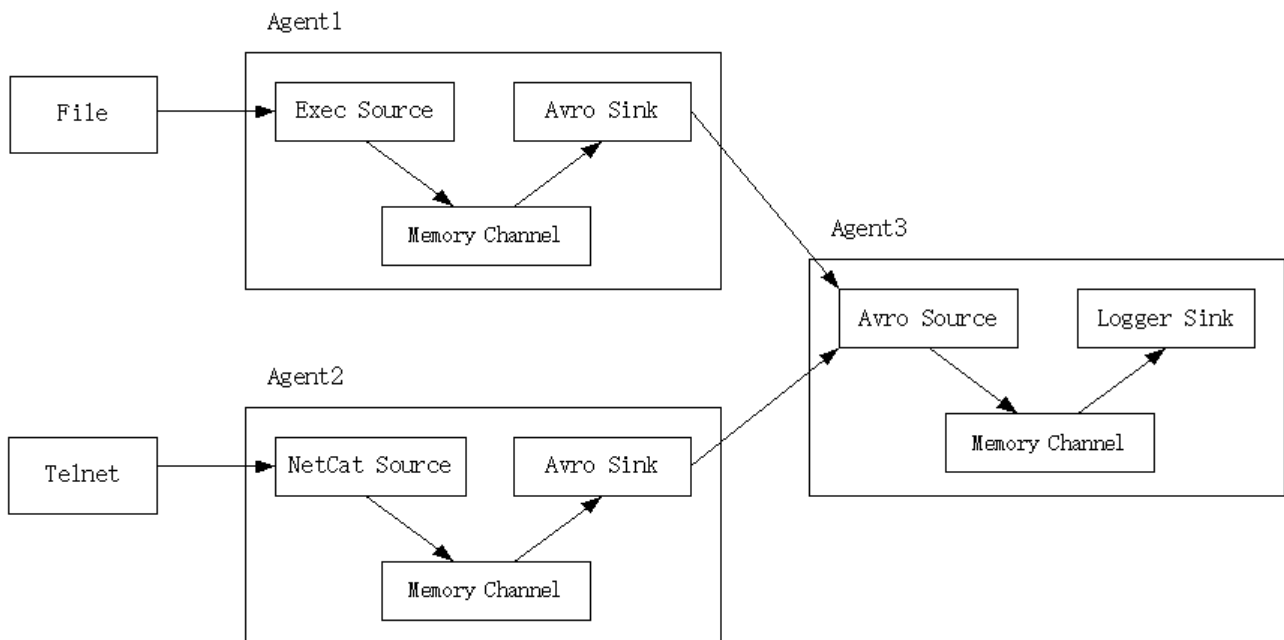
- Exec Source：实施监控一个文件的内容是否有增加；
- Avro Sink：用于跨节点传输，把捕捉到的数据传递给Agent3。

在Agent2中，各个组件的功能如下：

- NetCat Source：接收来自Telnet的数据
- Avro Sink：用于跨节点传输，把捕捉到的数据传递给Agent3

在Agent3中，各个组件的功能如下：

- Avro Source：接收来自Agent1和Agent2的数据；
- Logger Sink：把数据输出到控制台。



配置Flume

为Agent1创建一个配置文件multi-agent1.conf，其内容如下：

```
# multi-agent1.conf

# 设置各个组件名称
a1.sources = r1
a1.sinks = k1
a1.channels = c1

# 配置Source
a1.sources.r1.type = exec
a1.sources.r1.command = tail -F /home/hadoop/data.txt

# 配置Sink
a1.sinks.k1.type = avro
a1.sinks.k1.hostname = localhost
a1.sinks.k1.port = 4141

# 配置Channel
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 1000

# 把Source和Sink绑定到Channel
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

为Agent2创建一个配置文件multi-agent2.conf，其内容如下：

```
# multi-agent2.conf

# 设置各个组件名称
a2.sources = r1
a2.sinks = k1
a2.channels = c1

# 配置Source
a2.sources.r1.type = netcat
a2.sources.r1.bind = localhost
a2.sources.r1.port = 44444

# 配置Sink
a2.sinks.k1.type = avro
a2.sinks.k1.hostname = localhost
a2.sinks.k1.port = 4141

# 配置Channel
a2.channels.c1.type = memory
a2.channels.c1.capacity = 1000
a2.channels.c1.transactionCapacity = 1000

# 把Source和Sink绑定到Channel
a2.sources.r1.channels = c1
a2.sinks.k1.channel = c1
```

为Agent3创建一个配置文件multi-agent3.conf，其内容如下：

```
# multi-agent3.conf

# 设置各个组件名称
a3.sources = r1
a3.sinks = k1
a3.channels = c1

# 配置Source
a3.sources.r1.type = avro
a3.sources.r1.bind = localhost
a3.sources.r1.port = 4141
```

```
# 配置Sink
a3.sinks.k1.type = logger

# 配置Channel
a3.channels.c1.type = memory
a3.channels.c1.capacity = 1000
a3.channels.c1.transactionCapacity = 1000

# 把Source和Sink绑定到Channel
a3.sources.r1.channels = c1
a3.sinks.k1.channel = c1
```

执行过程

在前面小节，为了让Flume能够顺利访问HDFS，我们把相关的JAR包复制到了Flume安装目录下的lib子目录下，这会导致在“/usr/local/flume/lib”目录下存在一些重复的、不同版本的与netty库相关的JAR包，需要执行如下命令删除这些从Hadoop中复制过来的JAR包：

```
$ rm netty-3.10.6.Final.jar
$ rm *4.1.77*
```

现在可以启动Agent。需要先启动Agent3，再启动Agent2，最后启动Agent1，如果顺序不对，会出现拒绝连接的情况。

新建一个终端，执行如下命令启动Agent3：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file ./conf/multi-agent3.conf --name a3 -Dflume.root.logger=INFO,console
```

新建一个终端，执行如下命令启动Agent2：

```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file ./conf/multi-agent2.conf --name a2 -Dflume.root.logger=INFO,console
```

新建一个终端，执行如下命令启动Agent1：


```
$ cd /usr/local/flume
$ ./bin/flume-ng agent --conf ./conf --conf-file ./conf/multi-agent1.conf --name a1 -Dflume.root.logger=INFO,console
```

新建一个终端，在里面执行如下命令向data.txt文件中追加内容：

```
$ echo spark >> "/home/hadoop/data.txt"
$ echo hadoop >> "/home/hadoop/data.txt"
$ echo flume >> "/home/hadoop/data.txt"
```

新建一个终端（记作“NC窗口”），在里面执行如下命令：

```
$ nc localhost 44444
```

在NC窗口内输入如下内容：

```
i love spark
i love hadoop
i love flume
```

这时，在Agent3的终端内，就可以看到控制台输出的数据（如图所示）

```
2024-01-23 11:16:27,502 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{} body: 73 70 61 72 6B
    spark } ←
2024-01-23 11:16:36,504 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{} body: 68 61 64 6F 6F 70
    hadoop } ←
2024-01-23 11:16:45,907 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{} body: 66 6C 75 6D 65
    flume } ←
2024-01-23 11:17:07,911 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{} body: 69 20 6C 6F 76 65 20
    73 70 61 72 6B
    i love spark } ←
2024-01-23 11:17:12,322 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{} body: 69 20 6C 6F 76 65 20
    68 61 64 6F 6F 70
    i love hadoop } ←
2024-01-23 11:17:21,324 (SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO - org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)] Event: { headers:{} body: 69 20 6C 6F 76 65 20
    66 6C 75 6D 65
    i love flume } ←
```